Chapter 7 解答

CH07-Q01

1.
$$S(8) = \frac{1}{0.1 + 0.9/8} = \frac{1}{0.2125} = 4.70588,$$
$$E(8) = 4.70588/8 = 0.588235 = 58.8235\%.$$

2. $S_\infty = 1/0.1 = 10$。這是 asymptote，不是 $N = 8$ 或任何 finite processor count 的 result。

3.
$$S(128) = \frac{1}{0.03 + 0.97/128} \approx 26.61123.$$

4. 同為 $N = 8$、serial 降半：
$$S = \frac{1}{0.05 + 0.95/8} \approx 5.92593.$$

serial 仍 10%、workers 加倍：
$$S(16) = \frac{1}{0.1 + 0.9/16} = 6.4.$$

此例 finite comparison 是 $6.4 > 5.92593$。兩個模型的無限上界 20 與 10 仍不能拿來與 finite result 混排。

5. 由反解式：
$$f \geq \frac{N(S-1)}{S(N-1)} = \frac{100(89)}{90(99)} \approx 0.998878.$$

Serial fraction 是
$$1 - f \approx 0.001122 = 0.1122\%.$$

CH07-Q02

1. 重 worker 是 $0.02 \times 10000 = 200$ units。其他 99 workers 共 9800 units：
$$9800/99 \approx 98.99.$$

Parallel phase time 是 $\max(200, 98.99) = 200$，加 10 units overhead：
$$T = 210.$$

2. Sequential baseline $10000 + 10 = 10010$。Speedup：
$$S = 10010/210 \approx 47.67.$$

Efficiency：
$$E = 47.67/100 \approx 47.7\%.$$

Chapter 7 解答 （續）

3. 5% 重 worker 是 500 units，其他 99 分 9500：

$$\max(9500/99, 500) + 10 \approx 510.$$
$$S = 10010/510 \approx 19.63, \qquad E \approx 19.6\%.$$

4. Work stealing 讓 idle workers 從其他 queues 取得 tasks，可改善 skew 與 dynamic imbalance。新增 overhead 包含跨 worker 的 task metadata/atomic 操作，以及 moved task/data 的 cache/communication traffic；過細的 stealing 反而降低效能。

CH07-Q03

1.

$$S_4 = 300/80 = 3.75, \qquad E_4 = 3.75/4 = 93.75\%.$$

2. 兩次執行具有相同 per-core work，因此弱擴展效率為

$$E_{\text{weak}} = \frac{T_1(W)}{T_4(4W)} = \frac{75}{90} = \frac{5}{6} \approx 83.33\%.$$

加入題目所述線性單核成本假設，$T_1(4W) = 4(75) = 300$ s，所以

$$S_{\text{scaled}} = \frac{300}{90} = \frac{10}{3} \approx 3.333, \quad S_{\text{scaled}}/4 = \frac{5}{6}.$$

弱擴展效率比較不同總規模、相同 per-core work 的完成時間；scaled speedup 則比較相同較大問題的單核與四核時間。$90/75$ 是時間膨脹率，不是加速比；也不能把弱擴展效率再除一次4。

3. 小問題：$T_1 = 10t + 100t = 110t$。

$P = 10$：

$$T_{10} = 10t + 100t/10 = 20t, \quad S = 110/20 = 5.5.$$

$P = 100$：

$$T_{100} = 10t + 100t/100 = 11t, \quad S = 110/11 = 10.$$

大問題：$T_1 = 10t + 10{,}000t = 10{,}010t$。

$$T_{10} = 10t + 1{,}000t = 1{,}010t, \quad S = 10{,}010/1{,}010 \approx 9.91.$$
$$T_{100} = 10t + 100t = 110t, \quad S = 10{,}010/110 = 91.$$

Chapter 7 解答 （續）

4. 小問題的序列 10t 固定，當 parallel part 從 100t 分散到更多 cores 後，它很快不再是主體；分母被 serial part 壓住。大問題的 parallel work 隨 problem size 增加，因此更能攤薄固定 serial cost。這正是 weak scaling 的用途，也說明 strong 與 weak results不能互換。
5. 不能。Memory bandwidth 飽和、load imbalance、synchronization、communication、cache contention、frequency throttling 與 benchmark noise 都會降低 efficiency；data race 是 correctness defect，需另外驗證。

CH07-Q04

1. System U 是 UMA shared-memory multiprocessor；System M 是 private-address-space cluster/message-passing system。
2. 需要同一地點的 value correctness（coherence）與跨 thread publication ordering（acquire/release、fence 或 lock）。Cache coherence 只處理同一位置與 cached copy，不能自動讓 plain data store 排在 `ready` store 前，也不能消除 data race。
3. 只有當 acquire load **實際讀到該 release store 所寫的 1**，才建立 synchronizes-with。Writer program order 把 `data=42` 排在 release 前；reader program order 把 `r=data` 排在 acquire 後；三者形成 happens-before chain，因此 reader 的 `r` 是 42。

若 acquire 讀到 0，並沒有讀到那份 release value，就不能用這組 release/acquire 宣稱後續 data load 已被排序。若控制流仍進入讀取、或用 plain flag/pointer 作為 publication，程式有 data race，結果不能由這個 synchronizes-with 保證。

4. 題目第4小題是獨立的全 atomic-relaxed 版本，不是第3小題的有效 release/acquire 程式。可允許的觀察為：

```
writer: data.store(42, relaxed)
writer: ready.store(1, relaxed)
reader: ready.load(relaxed) returns 1
reader: data.load(relaxed) returns initial 0
```

每個 atomic object 的存取都不可撕裂，並各自遵守 modification order，故不是缺少 atomicity。缺的是跨 objects 的 synchronizes-with；writer 的程式順序沒有藉由 relaxed flag 傳遞成跨 thread happens-before，不能強制後面的 data load 看到42。換成第3小題的 release store 與實際讀到它的 acquire load，才排除此觀察。

若 data/ready 是 plain non-atomic，並有未排序的衝突讀寫，C/C++ 行為未定義。硬體的可見性圖只能說明同步需求，不能把該語言程式的全部行為限定為0或42。

Chapter 7 解答 （續）

5. Private address spaces 不能直接傳可移植 pointer；sender 要 marshal payload，transport 要 buffer、route，receiver 要 decode。可靠 transport 的 ACK 只證明 message 依協議到達，不能修復 receiver application 內另一組未排序 shared accesses。

CH07-Q05

1. Fine-grained 每 cycle 輪流 threads，能隱藏短與長 stalls，但個別 thread 被穿插。Coarse-grained 在 long stall 才切換，正常路徑對單 thread impact 小，但短 stall 可能難隱藏。SMT 在同一 issue cycle 從多 threads 填 slots，適合 multiple-issue core 的 ILP hole。
2. 不保證。Fine-grained 增加 per-thread elapsed time，雖然 aggregate throughput 可能提高。Single thread 可能比沒有 sharing 時更慢。
3. SMT 讓多 threads 共用 functional units，提高 issue slot occupancy；但多 threads 也共享 L1/L2/TLB/bandwidth，working sets 互相 evict，使 each thread 或其他 thread 的 hit rate 下降。
4. 若 single-thread ILP 已足以把 4 slots 填滿，SMT 的 cycle-level fillup 收益有限；主要效益可能在 thread stalls 時仍維持 issue，或提高 throughput 對 latency 的容錯。應量測 per-thread fairness、cache/TLB miss、bandwidth、power、thread interference 與 tail latency。

CH07-Q06

案例與成本類別	操作數	基準cycles/操作	基準時間cycles	加速後時間cycles
A：可向量化	80	1	80	80/8 = 10
A：不可向量化	20	10	200	200
A：合計	100	不同成本	280	210
B：可向量化	80	1	80	10
B：不可向量化	20	1	20	20
B：合計	100	等成本	100	30

1. 案例A的可加速時間比例是 $f = 80/280 = 2/7$，不是操作比例$80/100$。由實際成本相加，

$$S_A = \frac{280}{80/8 + 200} = \frac{280}{210} = \frac{4}{3}.$$

Chapter 7 解答 （續）

代入Amdahl也得到$1/[(1-2/7)+(2/7)/8]=4/3$。誤把操作比例0.8當時間比例會算成3.333倍，較正確結果高2.5倍。

2. 案例B的等成本前提使操作比例恰等於時間比例，因此$S_B=100/(80/8+20)=10/3$。有效寬度6時，$S_{B,6}=100/(80/6+20)=3$。若案例A也改有效寬度6，則是$280/(80/6+200)=1.3125$，不能答3。
3. 向量長度指定實際處理的元素數，硬體可用不同lane數分批執行；尾端以長度或mask限制有效元素。固定寬度SIMD通常要組成寬度相符的區塊並另處理尾端。
4. A的不可向量化200 cycles、B的20 cycles均不因加寬而消失。A寬度16時只有$280/(5+200)\approx 1.36585$，並非$2(4/3)$。記憶體頻寬、gather/scatter、設定、對齊、相依與降頻都可能使實際結果更低。

CH07-Q07

1. 4×4 mesh：

$$\text{horizontal}=4(4-1)=12,\quad \text{vertical}=4(4-1)=12,$$
$$|E_{\text{mesh}}|=24.$$

Torus 保留 24 條 local links，再加上 4 條 horizontal wraps 與 4 條 vertical wraps：

$$|E_{\text{torus}}|=24+8=32.$$

2. Mesh 的 degree sum 是 $2|E|=48$，平均 degree 3。Corner degree=2、edge node degree=3、interior degree=4。任兩 nodes 最多走水平和 3 hops 加垂直 3 hops：

$$D_{\text{mesh}}=3+3=6.$$

以 columns 0–1 與 2–3 分割，只有 4 條 horizontal local links 跨越，因此 bisection=4。

Torus 每個 node 左右上下各一 link，degree=4。每維最多走

$$\min(|\Delta c|,4-|\Delta c|)\le 2$$

hops，兩個維度合計 diameter=4。相同左右切面要計 4 條 local links 與 4 條 horizontal wrap links，因此 bisection=8；對所有 balanced cuts 取最小仍是 8。

3. Horizontal wraps：

$$(0,0)\leftrightarrow(3,0),\quad (0,1)\leftrightarrow(3,1),$$
$$(0,2)\leftrightarrow(3,2),\quad (0,3)\leftrightarrow(3,3).$$

Vertical wraps：

$$(0,0)\leftrightarrow(0,3),\quad (1,0)\leftrightarrow(1,3),$$
$$(2,0)\leftrightarrow(2,3),\quad (3,0)\leftrightarrow(3,3).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 7 解答 （續）

若刪掉 e.g. $(0,0)$ 到 $(3,0)$，corner $(0,0)$ 只剩上下與右側 local links，degree 變為 3；這已不是本題 degree-4 torus。

4. K_{16}：

$$|E| = \binom{16}{2} = 120, \quad \deg = 15, \quad D = 1.$$

任一 balanced cut 是 8×8 的 endpoint pairs，所以 bisection links=64。若 single-direction rate 為 b：

$$B_{\text{bisection,one}-\text{way}} = 64b.$$

5. Crossbar 的 endpoint 通常各接一個 switch port，degree 不是 15。內部 crosspoints 約 N^2，但 switch link 與 endpoint degree 是不同資源；不能把 K_{16} 的 64-cut links 直接當成 crossbar endpoint bandwidth。
6. Bisection 是 uniform balanced traffic 的 worst-cut 摘要。Sparse-matrix application 可能有 hot nodes、局部化 access、directional congestion 或 small messages；routing、flow control、message size 與 physical link rate 都會讓 delivery time不同。

CH07-Q08

1. 令 $Q = F/I$。Memory time $Q/B = F/(IB)$，compute time F/P_C。若能重疊：

$$T \geq \max(F/P_C, F/(IB)), \qquad P = F/T \leq \min(P_C, IB).$$

折點由 $P_C = IB$ 得 $I_{\text{ridge}} = P_C/B$。本題：

$$I_{\text{ridge}} = 200/40 = 5 \text{ FLOP/byte}.$$

2. Kernel A：

$$IB_D = 0.4(40) = 16, \qquad P_A \leq \min(200, 16) = 16 \text{ GFLOP/s},$$

memory-bound。Kernel B：

$$IB_D = 12(40) = 480, \qquad P_B \leq \min(200, 480) = 200 \text{ GFLOP/s},$$

compute-bound。

3. 分層強度：

$$I_D = \frac{10^9}{0.25 \times 10^9} = 4 \text{ FLOP/byte},$$

$$I_S = \frac{10^9}{4 \times 10^9} = 0.25 \text{ FLOP/byte}.$$

DRAM bound：

$$I_D B_D = 4(100) = 400 \text{ GFLOP/s}.$$

Chapter 7 解答 （續）

Shared bound：

$$I_S B_S = 0.25(1000) = 250 \text{ GFLOP/s}.$$

Compute time：

$$10^9/(1000 \times 10^9) = 0.001 \text{ s} = 1 \text{ ms}.$$

4. 跨層：

$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s}.$$

時間：

$$T_D = 0.25/100 = 0.0025 \text{ s} = 2.5 \text{ ms},$$
$$T_S = 4/1000 = 0.004 \text{ s} = 4 \text{ ms},$$
$$T \ge \max(1, 2.5, 4) = 4 \text{ ms}.$$

在 DRAM 自己那張圖，ridge 是 $1000/100 = 10$，工作點 $(4, 400)$ 位於斜線；在 shared 圖，ridge 是 $1000/1000 = 1$，工作點 $(0.25, 250)$ 也位於斜線。Shared roof 的 plateau 仍是 1000；250 是把 compute、DRAM 與 shared 三層合併後的上界。最終工作點若沿同一 $I_D = 4$ 表示，可以是 $(4, 250)$，但 shared 的獨立橫座標仍是 $I_S = 0.25$。

5. 不能。Roofline 只給 upper bound；30 GFLOP/s 可能受 dependency chain、instruction mix、cache hit/miss traffic、coalescing、occupancy、load imbalance、clock/power 或 benchmark 只量到部分 phase 影響。應先讀 profiler 的 memory sectors、cache/TLB、stall reason 與 per-SM utilization，再對照 model inputs。

Chapter 7 解答 （續）

原創驗算程式附錄: performance_models.py

```
"""Re-runnable Amdahl, load-balance, topology, and roofline examples."""


def amdahl(parallel_fraction: float, workers: int) -> float:
    return 1.0 / ((1.0 - parallel_fraction) + parallel_fraction / workers)


def efficiency(speedup: float, workers: int) -> float:
    return speedup / workers


def load_balanced_time(total_work: float, workers: int, heavy_fraction: float, overhead:
float) -> float:
    heavy = total_work * heavy_fraction
    remaining = total_work - heavy
    return max(heavy, remaining / (workers - 1)) + overhead


def roofline(peak_gflops: float, bandwidth_gb_s: float, intensity: float) -> tuple[float,
float]:
    memory_roof = bandwidth_gb_s * intensity
    return memory_roof, min(peak_gflops, memory_roof)


def mesh_metrics(width: int) -> tuple[int, int, int]:
    return 4, 2 * (width - 1), width


def main() -> None:
    speedup8 = amdahl(0.90, 8)
    assert abs(speedup8 - 4.705882352941177) < 1e-12
    assert abs(efficiency(speedup8, 8) - 0.5882352941176471) < 1e-12

    elapsed2 = load_balanced_time(10000, 100, 0.02, 10)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 7 解答 （續）

```
    elapsed5 = load_balanced_time(10000, 100, 0.05, 10)
    assert elapsed2 == 210
    assert abs(elapsed5 - 510) < 1e-12
    assert abs((10010 / elapsed2) - 47.666666666666664) < 1e-12

    degree, diameter, bisection = mesh_metrics(4)
    assert (degree, diameter, bisection) == (4, 6, 4)

    ridge = 200 / 40
    memory_a, bound_a = roofline(200, 40, 0.4)
    memory_b, bound_b = roofline(200, 40, 12)
    memory_c, bound_c = roofline(200, 40, 2)
    assert ridge == 5
    assert (memory_a, bound_a) == (16, 16)
    assert (memory_b, bound_b) == (480, 200)
    assert (memory_c, bound_c) == (80, 80)

    print(f"Amdahl f=0.90, N=8: speedup={speedup8:.3f}, efficiency={efficiency(speedup8,
8):.3f}")
    print(f"load imbalance 2%: {elapsed2:.0f} time units; 5%: {elapsed5:.0f}")
    print("4x4 mesh: max degree=4, diameter=6, bisection links=4")
    print(f"roofline ridge={ridge:.1f} FLOP/byte")
    print(f"kernel A: memory roof={memory_a:.1f}, bound={bound_a:.1f} GFLOP/s")
    print(f"kernel B: memory roof={memory_b:.1f}, bound={bound_b:.1f} GFLOP/s")
    print(f"kernel C: memory roof={memory_c:.1f}, bound={bound_c:.1f} GFLOP/s")
    print("multicore checks: PASS")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 7 解答 （續）

原創驗算程式附錄: revision_models.py

```
"""Cost, hierarchy, stack, and coherence models used by the revised manuscripts."""

from dataclasses import dataclass


@dataclass(frozen=True)
class OperationClass:
    count: int
    baseline_cycles_each: float
    local_speedup: float


def cost_breakdown(classes: tuple[OperationClass, ...]) -> tuple[list[float], list[float]]:
    before = [item.count * item.baseline_cycles_each for item in classes]
    after = [time / item.local_speedup for time, item in zip(before, classes)]
    return before, after


@dataclass(frozen=True)
class MemoryBoundary:
    name: str
    bytes_transferred: float
    bytes_per_second: float


def hierarchical_bound(flops: float, compute_flops_per_second: float, boundaries:
tuple[MemoryBoundary, ...]) -> dict:
    levels = [{"name": level.name, "bytes": level.bytes_transferred,
               "intensity": flops / level.bytes_transferred,
               "bound_flops_per_second": flops / level.bytes_transferred *
level.bytes_per_second,
               "minimum_seconds": level.bytes_transferred / level.bytes_per_second}
              for level in boundaries]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 7 解答 （續）

```
    compute_seconds = flops / compute_flops_per_second
    return {"levels": levels, "bound_flops_per_second": min(compute_flops_per_second, *
(level["bound_flops_per_second"] for level in levels)),
            "minimum_seconds": max(compute_seconds, *(level["minimum_seconds"] for level in
levels))}


def p1_cases() -> dict:
    mixed = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 10, 1)))
    equal = cost_breakdown((OperationClass(80, 1, 8), OperationClass(20, 1, 1)))
    base = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", 4e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    tiled = hierarchical_bound(1e9, 1e12, (MemoryBoundary("DRAM", .25e9, 100e9),
MemoryBoundary("shared", 4e9, 1000e9)))
    return {"mixed_cost": {"before": mixed[0], "after": mixed[1], "speedup":
sum(mixed[0])/sum(mixed[1])},
            "equal_cost": {"before": equal[0], "after": equal[1], "speedup":
sum(equal[0])/sum(equal[1])},
            "untiled": base, "tiled": tiled,
            "wrong_reused_dram_intensity_bound": min(1e12, 4 * 1000e9)}


if __name__ == "__main__":
    import json
    result = p1_cases()
    assert result["mixed_cost"]["before"] == [80, 200]
    assert result["mixed_cost"]["after"] == [10, 200]
    assert abs(result["mixed_cost"]["speedup"] - 4/3) < 1e-12
    assert abs(result["equal_cost"]["speedup"] - 10/3) < 1e-12
    assert result["untiled"]["bound_flops_per_second"] == 25e9
    assert result["tiled"]["bound_flops_per_second"] == 250e9
    assert result["tiled"]["minimum_seconds"] == .004
    assert result["wrong_reused_dram_intensity_bound"] == 1e12
    print(json.dumps(result, indent=2))
    print("revision cost and hierarchical traffic models: PASS")
```

參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study